Parallel & Distributed Computing - Milestone 1
Distributed Web Crawler & Page Rank

Web Crawler Speed Comparison

Sequential – Ray Master-Worker – Ray Peer-to-Peer

Hamza Ahmed - 29265
Muhammad Hamza Arif - 29234
Laraib Fatima Farooqui - 26521
Arbaaz Murtaza - 29052

Overview:

This document benchmarks three distinct web crawling architectures in order to evaluate their performance characteristics across hyperparameters. The goal is to understand how parallelism (via Ray) affects speed, throughput, and scalability compared to a simple single-process approach, and how performance scales with increasing workers, seeds, and depth.
Framework: Ray (distributed computing) + aiohttp
Dataset: Majestic Million top .com domains
Metrics: Elapsed time, throughput, URLs crawled, graph edges

Architectures Compared:

Architecture	Description	Parallelism
Sequential	Single-process async BFS crawler using aiohttp. All states in plain Python dicts/sets.	None (1 process)
Master-Worker	Centralised Ray actor (MWMaster) holds the global queue. Worker actors pull batches and report results back.	Ray actors: 4, 8, or 12 workers

Peer-to-Peer	Fully autonomous worker actors. Each owns a URL partition via consistent hashing; workers route discovered links directly to peers.	Ray actors: 4, 8, or 12 workers

Hyperparameters:

Each architecture is evaluated over the following grid of hyperparameters in order to build a comprehensive understanding of performance:

Parameter	Values Tested	Applies To
num_seeds	10, 20, 40	All three crawlers
depth	1, 2, 3	All three crawlers
num_workers	4, 8, 12	Ray crawlers only (Master-Worker, P2P)

Code Architecture:

Shared Utilities

All three crawlers share a common utility layer:

- normalize_url(url): Strips www prefix, lowercases netloc, removes trailing slashes, strips query strings and fragments for deduplication.
- get_base_domain(url): Extracts the registered domain from a URL for per-domain rate limiting.
- hash_id(link, n_workers): MD5-based consistent hash to assign URLs to a specific worker partition (used by P2P).
- make_ssl_ctx(): Returns an SSL context with certificate verification disabled (for crawling robustness).
- load_seeds(n, csv_path): Samples n '.com' domains from the Majestic Million CSV; reproducible via random_state=42.

Sequential Crawler
The Class 'SequentialCrawler' implements a BFS async crawler within a single Python process, which:

- Uses aiohttp for async HTTP requests with a 10-second timeout.
- Parses HTML with selectolax (faster than BeautifulSoup).
- Enforces a per-domain cap (max_per_domain=50) to avoid crawling a single site exhaustively.
- Processes URLs in batches of 50 concurrently using asyncio.gather.
- Maintains visited, domain_counts, and graph as plain Python data structures.

Ray Master-Worker Crawler
This is implemented via the actors, MWMaster (coordinator) and MWWorker (fetchers), in which:

- MWMaster is a Ray remote actor that centralises all state: the BFS queue, the visited set, the graph, and the domain counts.
- Workers pull batches from the master using get_batch(), fetch the URLs asynchronously, and then submit the results with submit_results().
- Workers use max_concurrency=2 to allow overlapping fetch cycles.
- The master serialises all state updates, which prevents race conditions but can become a bottleneck at high worker counts.

Ray Peer-to-Peer Crawler
This is implemented via the actors, P2PSignal (termination coordinator) + P2PWorker (autonomous crawlers), in which:

- Each worker is assigned a partition of the URL space via hash_id (consistent hashing on the URL string).
- Workers route newly discovered links directly to the responsible peer using Ray remote calls, eliminating the central bottleneck.
- A P2PSignal actor implements a graceful countdown-based termination: when any worker exhausts its queue, it starts a timer; all others check the signal and stop when time elapses.
- Per-domain concurrency is regulated by asyncio.Semaphore(3) within each worker.
- max_concurrency=100 is set on each worker actor to support high async throughput.

Metrics:

For every hyperparameter configuration and crawler, the following four metrics are recorded:

Metric	Description	Unit
elapsed_sec	Wall-clock time from crawl start to finish	Seconds

urls_crawled	Total number of unique URLs successfully fetched	Count
total_edges	Number of directed links in the crawled web graph	Count
throughput	urls_crawled / elapsed_sec — primary performance indicator	URLs/second

---

Results:

Sequential Crawler

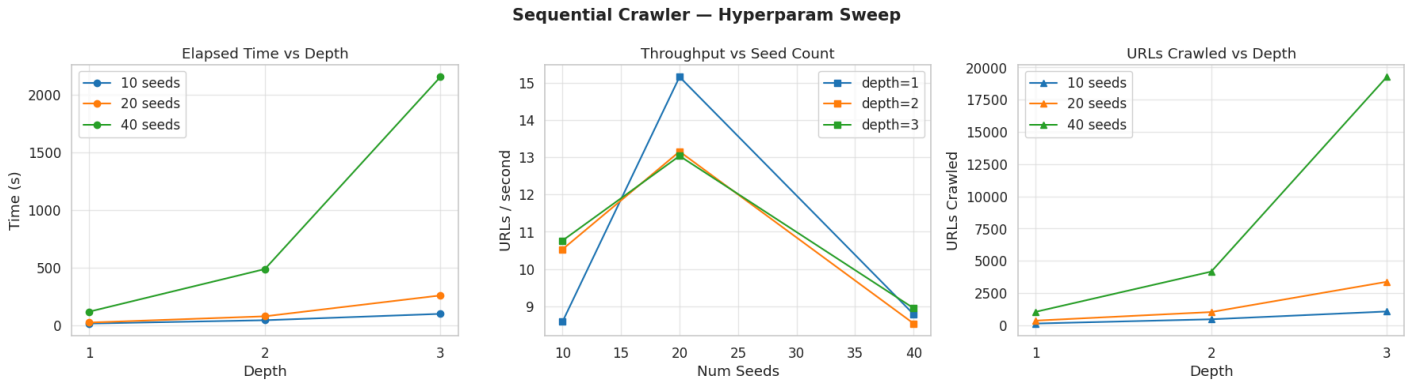
Seeds	Depth	Workers	Time (s)	URLs Crawled	Edges	URLs/s
10	1	1	14.8	127	4,615	8.6
10	2	1	43.4	457	10,888	10.5
10	3	1	98.7	1,062	27,111	10.8
20	1	1	23.3	353	16,921	15.2
20	2	1	77.4	1,018	32,233	13.2
20	3	1	258.3	3,367	99,169	13.0

40	1	1	117.8	1,035	45,481	8.8
40	2	1	487.1	4,158	162,588	8.5
40	3	1	2,153.7	19,268	814,048	8.9

Observations:

- At depth=3 with 40 seeds, elapsed time reaches 2,153 seconds i.e. nearly 36 minutes, which is the heaviest single run.
- Throughput stays relatively stable at 8 - 15 URLs/second regardless of configuration, showing the single-process ceiling.
- The 20-seed, depth=1 configuration delivers the highest sequential throughput at 15.2 URLs/s.
- Graph edges scale aggressively with depth, going from 4,615 at 10 seeds and depth=1 to 814,048 at 40 seeds and depth=3.

Performance Charts:



Ray Master-Worker Crawler

Seeds	Depth	Workers	Time (s)	URLs Crawled	Edges	URLs/s	vs Seq

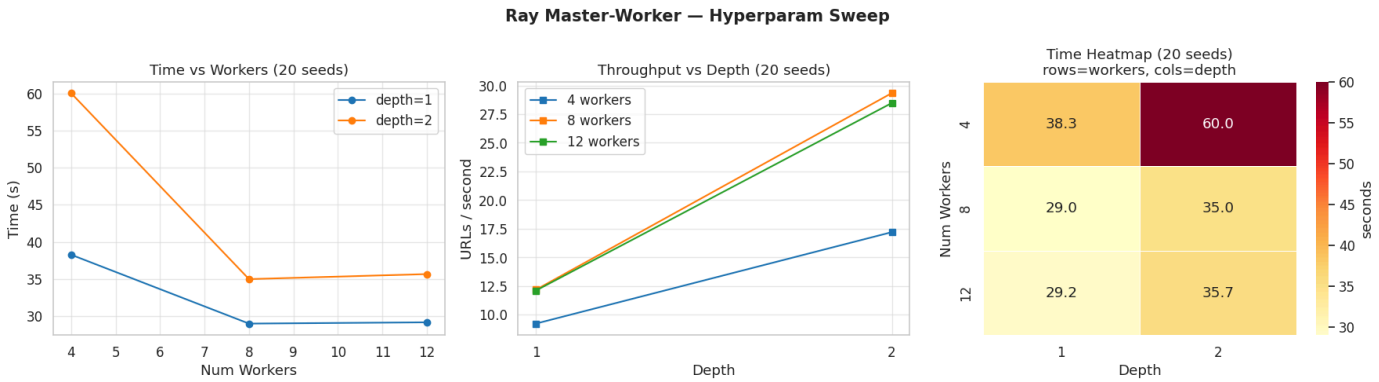
10	1	4	37.7	127	4,611	3.4	−61% slower
10	1	8	26.6	127	4,611	4.8	−45% slower
10	1	12	27.0	127	4,611	4.7	−46% slower
10	2	4	46.7	451	10,003	9.7	−8% slower
10	2	8	36.6	451	10,868	12.3	+17% faster
10	2	12	35.6	451	11,297	12.7	+21% faster
20	1	4	38.3	353	21,667	9.2	−39% slower
20	1	8	29.0	353	18,758	12.2	−20% slower
20	1	12	29.2	353	16,880	12.1	−20% slower
20	2	4	60.0	1,033	37,848	17.2	+31% faster
20	2	8	35.0	1,026	35,794	29.3	+122% faster

20	2	12	35.7	1,015	34,632	28.5	+116% faster
40	1	4	114.9	1,019	55,085	8.9	+1% faster
40	1	8	80.4	991	51,496	12.3	+40% faster
40	1	12	70.0	991	51,605	14.2	+62% faster
40	2	4	328.3	4,148	164,423	12.6	+48% faster
40	2	8	141.8	4,094	164,889	28.9	+239% faster
40	2	12	93.5	4,189	162,514	44.8	+425% faster

Observations:

- At low workloads (10 seeds, depth 1), Master-Worker is slower than Sequential.The overhead from the Ray actor dominates when there is little work to distribute.
- At higher workloads (40 seeds, depth 2, 12 workers), throughput reaches 44.8 URLs/s, which is a 425% improvement over sequential.
- Scaling from 4 to 8 workers consistently gives the biggest throughput jump; going from 8 to 12 shows diminishing returns.
- The best single configuration is 40 seeds, depth 2, 12 workers: 44.8 URLs/s.

Performance Charts:



Ray Peer-to-Peer Crawler

Seeds	Depth	Workers	Time (s)	URLs Crawled	Edges	URLs/s	vs Seq
10	1	4	11.2	10	117	0.9	−90% slower
10	1	8	14.3	31	958	2.2	−74% slower
10	1	12	14.5	72	2,833	5.0	−42% slower
10	2	4	20.7	92	2,376	4.4	−58% slower
10	2	8	52.3	333	21,507	6.4	−39% slower
10	2	12	29.1	415	11,666	14.3	+36% faster

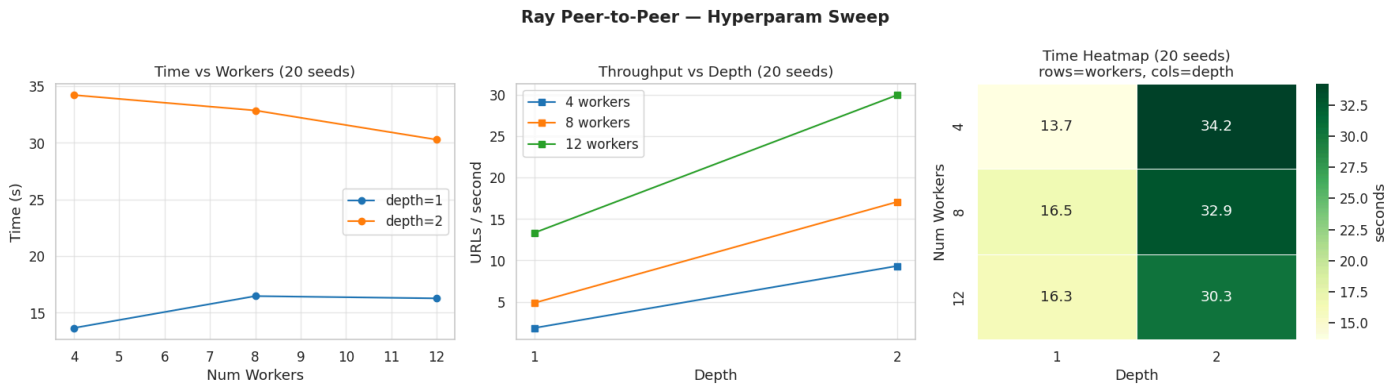
20	1	4	13.7	25	556	1.8	−88% slower
20	1	8	16.5	80	2,028	4.9	−68% slower
20	1	12	16.3	217	8,347	13.3	−12% slower
20	2	4	34.2	319	10,630	9.3	−29% slower
20	2	8	32.9	561	17,049	17.1	+30% faster
20	2	12	30.3	908	36,757	30.0	+128% faster
40	1	4	89.0	236	14,467	2.7	−69% slower
40	1	8	51.2	352	16,724	6.9	−21% slower
40	1	12	52.0	339	24,332	6.5	−26% slower
40	2	4	301.1	1,905	49,486	6.3	−26% slower
40	2	8	301.1	2,220	74,127	7.4	−13% slower

40	2	12	184.2	2,321	174,802	12.6	+48% faster

Observations:

- P2P is the weakest performer at low worker counts. Without a central queue, many URLs are perhaps missed or left unvisited, which explains the very low number of URLs crawled at 4 workers.
- Performance improves significantly as the number of workers increases. At 12 workers, the P2P crawler becomes competitive.
- The best P2P configuration (20 seeds, depth=2, 12 workers) achieves 30.0 URLs/s, which is faster than sequential but slower than the Master-Worker at roughly the same setting (28.5 vs 30.0).

Performance Charts:



Summary:

- At depth=1 with small seed counts (10 to 20 seeds), Sequential outperforms both the Ray Master-Worker and Peer-to-Peer approaches. Master-Worker takes 29.2s vs Sequential's 23.3s at 20 seeds, meaning Ray overhead costs more than the parallelism gains at low workloads.
- At depth=2, both Ray approaches work faster, with the Master-Worker being 2.2x faster and Peer-to-Peer being 2.6x faster than the Sequential approach.
- Going from 4 to 8 workers gives the biggest throughput jump in both Ray architectures by roughly doubling throughput in most configurations. On the other hand, going from 8 to 12 shows diminishing returns, especially in Master-Worker, because the central master actor becomes a bottleneck when workers queue up to submit results.

- At the heaviest config (40 seeds, depth=2, 12 workers), Master-Worker delivers a 5.1x speedup over Sequential (93.5s vs 487.1s), with the throughput jumping from 8.5 to 44.8 URLs/s.
- Ray Master Worker is the most consistent performer across configurations, as the central queue ensures that no URL is missed.
- Ray Peer-to-Peer crawls fewer URLs at lower worker counts due to hash-based partitioning leaving some workers with empty queues, but scales well at larger worker counts (12 workers).
- For large-scale crawls, i.e. with many seeds and high depth, Ray Master-Worker with 8 to 12 workers would be the optimal architecture. For shallow or small crawls, Sequential remains competitive.